The AI Fellowship

Session 3

AI That Does Things

Understanding AI Agents

Aryan | Robotics and AI Lead @Hexaware

Hosted by The AI Fellowship

What if AI didn't just answer questions—
what if it took action?

The question that separates a chatbot from an agent.

Agenda

01 **AI Agents**

What agents are, how the loop works, and what makes them different from regular AI

02 **n8n**

Build an AI-powered workflow without writing a single line of code

03 **Ultravox**

Talk to a voice agent live — in real time, in this room

These aren't three separate topics. They're three versions of the same idea.

How Most of Us Use AI Today

The current model — and why it matters

The Model

YOU		AI		ANSWER
"What's the capital of France?"	->	thinks for a moment	->	"Paris."

AI as a very smart encyclopedia.
Ask a question. Get an answer. Done.

Every step requires you. The AI doesn't move unless you prompt it.

Why It Matters

- Useful for one-shot questions — definitions, summaries, quick answers
- Breaks down on multi-step work — you become the connector between each AI response
- You are the automation — every step still requires a human to prompt it forward
- Scale is limited — you can only move as fast as you can type

This is incredibly useful. But it's still just one step.

What If AI Could Handle the Whole Thing?

Same AI. Now it has hands.

The Task

"Book me the cheapest flight to Paris this weekend under ₹20,000."

You give it the goal.
The agent handles the rest.

Same technology you use in ChatGPT. One key difference: it acts.

What the Agent Does

① Searches 3 flight comparison sites

② Filters by price and travel time

③ Opens the cheapest option

④ Fills in your details

⑤ Sends you a confirmation to approve

You're still in charge of the final call — the agent confirms before booking.

What Exactly Is an AI Agent?

"An AI agent is an AI that can use tools, make decisions, and take steps to complete a goal — without you doing each step yourself."

Tools

It can act on things

Search the web. Send an email. Read a file. Run a calculation.

You decide which tools it gets. It can only do what you've given it access to.

Tools are the agent's access pass.

Decisions

It chooses what to do next

It doesn't execute blindly. It looks at what it knows and decides which tool to use, in what order, and whether it has enough information yet.

The brain decides. You don't write the logic.

Steps

It keeps going until done

It works through the problem step by step without waiting for you between each one. It stops when the job is done, or when it needs your input.

It runs. You supervise.

Think of a highly organised colleague who runs a project for you — without being micromanaged.

Every Agent Is Built From Three Things

Brain

The reasoning engine

The LLM — the same technology that powers ChatGPT or Claude.

It reads everything available to it, reasons about what to do, and makes decisions.

You don't program the logic. You tell it what you want and it figures out the steps.

Brain = GPT, Claude, Gemini — your choice.

Tools

The actions it's allowed to take

Search the web. Send an email. Read a file. Run a calculation. Update a spreadsheet.

You decide which tools it gets. It can only do what you've given it permission to do.

Tools are the access pass. You control what it can touch.

Memory

What it remembers while working

What you've told it. What it has already done. What it found out.

Without memory, every step starts from scratch. With it, the agent builds on what it already knows.

Memory turns a one-shot tool into a working assistant.

Change any one of these and you get a different agent.

How an Agent Actually Works

The See → Think → Do → Check loop — used in every agent built today

1. SEE

What's the goal?
What's new?

2. THINK

What should I do next?
Which tool should I use?

3. DO

Use a tool.
Take action.

4. CHECK

Did that work?
Is the job done?

This loop repeats until the job is done — or until the agent needs your input.

Technical name: the ReAct loop (Reasoning + Acting). Used in almost every agent built today.

Agents Already Working in the Real World

Research Agent

TASK GIVEN

"Find 10 competitors and summarise their pricing."

WHAT IT DOES

- Searches the web for each competitor
- Reads the relevant pricing pages
- Compiles a structured comparison table
- Writes a plain-English summary

Time saved: ~3 hours

Support Agent

TASK GIVEN

"Handle tier-1 support tickets automatically."

WHAT IT DOES

- Reads the incoming support ticket
- Checks the customer's account history
- Replies if it can resolve the issue
- Escalates to a human if needed

Available: 24 / 7

Scheduling Agent

TASK GIVEN

"Set up a meeting with five stakeholders."

WHAT IT DOES

- Checks all five calendars for availability
- Proposes suitable time slots
- Sends calendar invites to all parties
- Confirms attendance and handles changes

Human effort: 0 minutes

These aren't science fiction. Teams are running all three today.

Tools Are What Let an Agent Act

You decide what it can touch — it can only do what you've given it access to

The Model

AI BRAIN

↓

Search the web	Send an email	Check a calendar
Read a file	Update a spreadsheet	Run code

It can only do what you've given it access to. You are always in control.

Why It Matters

- Tools are how the agent interacts with the real world — without them it can only think, not act
- You control the risk: don't connect it to email and it cannot send email. Don't give it file access and it cannot touch your files
- Start with one tool. See how it behaves. Add more when you're confident. You never have to hand over everything at once

A new hire can only use systems you've given them a login for. An agent is exactly the same.

Memory Is What Keeps the Agent Coherent

Short-Term Memory

What's happened in this session

Everything in the current conversation: messages sent, tools called, results received.

Like your own working memory during a meeting — active right now, gone when the session ends.

Without it, the agent would forget what it searched for two steps ago.

Resets when the session ends.

Long-Term Memory

Information stored and searchable later

A database the agent can write to and search — even in a future session.

If it researched your competitors last week and stored the findings, it can retrieve them today without searching again.

Like a filing cabinet it built and can always reach back into.

Persists across sessions.

Pinned Facts

Things you always want it to know

Your name. Your preferences. Your company's tools. The way you like documents formatted.

These get included every time the agent starts up — so you don't have to re-explain yourself in every session.

Always present, every session.

Without memory, every step starts from scratch. With it, the agent builds on what it already knows.

Agents Aren't Perfect. Here's What to Watch For.

STUCK IN LOOP	Keeps trying the same approach that isn't working. Calls the same tool five times hoping for a different result.	→	**Set a maximum number of attempts. After N tries with no progress, stop and ask the human.**
WRONG TOOL	Uses a web search when the answer is in the document you gave it — or vice versa.	→	**Write clearer tool descriptions. A better brief gets better choices. Vague descriptions get vague decisions.**
CONFIDENTLY WRONG	Hallucinates a step or invents a result. Acts certain when it should have asked for help.	→	**Keep humans in the loop for decisions that actually matter: send this email, approve this payment, delete this file.**

This is why agents are powerful assistants — not replacements for human judgement.

Sometimes One Agent Isn't Enough

Know when a single agent is right — and when to build a team

One Agent

✓ Simple, focused tasks

✓ Fast to set up and understand

✓ Easy to debug when something goes wrong

USE WHEN

The task is focused and the steps are clear. Research, summarise, send. One trigger, one goal, one output.

Example: Research agent, support agent, scheduling agent

Start here. Always. Add complexity only when you hit a real limitation.

Team of Agents

✓ Complex, multi-step tasks

✓ Parallel work across specialist roles

✓ Too large or too broad for a single agent

USE WHEN

The task is too big or complex for one agent. One manager delegates to specialists: search, write, review, verify.

Example: Deep research report

Search → Read → Summarise → Write

The best production agents are usually simpler than you'd expect.

Not All Agents Are the Same

Here's how to think about the range — and where most useful agents live

SCRIPTED (safe) → **SEMI-AUTONOMOUS (useful)** → **AUTONOMOUS (careful)**

Fully Scripted

AI follows a fixed script. No decisions.

Responds to triggers from a pre-set list. Completely predictable.

Cannot adapt to anything outside the script.

EXAMPLE

A FAQ chatbot with pre-written answers.

Predictable. But limited.

★ RECOMMENDED

Semi-Autonomous

AI decides within guardrails you set.

Handles the repetitive work — searching, drafting, organising.

You stay in the loop for decisions that actually matter. You set the guardrails.

EXAMPLE

An agent that researches and drafts — but you approve before it sends.

This is where almost all useful production agents live.

Fully Autonomous

AI decides everything independently.

Makes every decision without asking. Exists in research today.

Not where most people should start — or end up.

EXAMPLE

An agent that manages your calendar, email, and budget independently.

Powerful. Requires extreme care.

AI handles the repetitive work. You handle the judgement calls. That balance is the design philosophy behind every agent working well in production today.

Let’s Watch an Agent Think — Live

A LangChain ReAct agent with two tools: Web Search and Wikipedia

The Task

“Who founded OpenAI and what are each of the founders doing now?”

TOOLS AVAILABLE

Web Search

Wikipedia

I haven’t told it which tool to use for which part — it will decide that itself.

Don’t read the code. Watch the output. That’s where the agent is.

What to Watch For

Thought:	The agent deciding what to do next
Action:	The agent choosing and calling a tool
Observation:	The result coming back from the tool
Final Answer:	The agent deciding it has enough

SEE → **THINK** → **DO** → **CHECK**

The same loop — now you’ll see it run.

You Just Watched an Agent Work

Now let's build one — without writing a single line of code

What You Just Saw

```
from langchain import ...

agent = create_agent(
  tools=[search, wiki],
  llm=gpt4
)

executor.invoke(task)
```

What you just saw.

Powerful. But you need to know how to code to build it.

The engine room of a ship. Everything runs through here.

What You're About to Build

Schedule Trigger → **AI Agent** → **Slack Output**

What you're about to build.

Same agent. No code. Just connect the blocks.

If the code version is the engine room, n8n is the bridge. Same vessel. You just don't go below deck.

Same logic. Same loop. Different interface.

n8n — Visual Automation for Everyone

n8n is a visual builder for automated workflows. Connect blocks together. It handles the logic.

Connect Any Two Apps

400+ integrations built in

- Gmail + Notion
- Slack + Google Sheets
- WhatsApp + OpenAI
- HubSpot + Airtable

Whatever tools your team already uses, n8n can talk to them.

Run on Schedule or Trigger

Set it and forget it

- Every morning at 8am
- When a form is submitted
- When an email arrives
- When a Slack message appears

You set the condition. It takes care of the rest.

Use AI to Decide

Drop in an AI Agent node

- Give it tools
- Write a system prompt
- It reasons and acts
- No code. No setup.

The same ReAct loop — inside a block you drag and drop.

Free to start. 900+ ready-made templates. Runs in the cloud. n8n.io

Same Thing. Two Very Different Experiences.

Same workflow. One written in code, one built visually.

WRITING IT IN CODE

```
if email.subject
  .contains("urgent"):

  slack.send_message(
    channel="#alerts",
    text=email.body
  )
```

(And this is one of the simplest possible examples.)

✕ Hard to read if you didn't write it

✕ Easy to break; slow to debug

What a developer sees.

vs

BUILDING IT IN n8n

Email Trigger

↓

Check: subject = "urgent"?

↓

Send Slack Message

Three blocks. Drag. Connect. Done.

No code | **No debugger** | **Modify in minutes**

What you'll build today.

The logic is identical. The barrier to entry is not.

Three Types of Blocks. That's It.

Master these three and any workflow becomes readable

Trigger

ON EVENT

What starts the workflow

- New email arrives
- Form submitted
- Every day at 8am
- Slack message received

Nothing runs until a trigger fires.

Action

DO SOMETHING

What the workflow does

- Send a message
- Add a row to a spreadsheet
- Call an AI model
- Update a CRM record

Actions are the work. One block, one job.

Decision

BRANCH LOGIC

Branch the logic

- If urgent → notify manager
- If not urgent → archive
- If AI says yes → proceed
- If error → alert team

Decisions let the workflow handle different situations without you.

Every workflow — no matter how complex — is some combination of these three.

This Is Where It All Connects

The AI Agent node IS the ReAct loop — running inside a block you can drag and drop

Gmail Trigger

↓

AI AGENT NODE

System Prompt:

```
You are a research assistant. Summarise the email and find
relevant background information. Keep your output concise and
factual.
```

Tools: Web Search | Wikipedia

↓

Slack — Send Output

1 The system prompt is the configuration

A few sentences of plain English. That's it. No logic flows. No decision trees.

2 This IS the ReAct loop

See → Think → Do → Check. The same loop from Slide 8 — now running inside a block you drag onto a canvas.

3 You're writing instructions, not code

If you can brief a smart colleague, you can write a system prompt. That's the whole job.

You're not configuring an algorithm. You're writing instructions — in plain English.

Same Thing. Two Very Different Experiences.

Same workflow. One written in code, one built visually.

WRITING IT IN CODE

```
if email.subject
  .contains("urgent"):

  slack.send_message(
    channel="#alerts",
    text=email.body
  )
```

(And this is one of the simplest possible examples.)

✕ Hard to read if you didn't write it

✕ Easy to break; slow to debug

What a developer sees.

VS

BUILDING IT IN n8n

Email Trigger

↓

Check: subject = "urgent"?

↓

Send Slack Message

Three blocks. Drag. Connect. Done.

No code | No debugger | Modify in minutes

What you'll build today.

The logic is identical. The barrier to entry is not.

Three Types of Blocks. That's It.

Master these three and any workflow becomes readable

Trigger

ON EVENT

What starts the workflow

- New email arrives
- Form submitted
- Every day at 8am
- Slack message received

Nothing runs until a trigger fires.

Action

DO SOMETHING

What the workflow does

- Send a message
- Add a row to a spreadsheet
- Call an AI model
- Update a CRM record

Actions are the work. One block, one job.

Decision

BRANCH LOGIC

Branch the logic

- If urgent → notify manager
- If not urgent → archive
- If AI says yes → proceed
- If error → alert team

Decisions let the workflow handle different situations without you.

Every workflow — no matter how complex — is some combination of these three.

This Is Where It All Connects

The AI Agent node IS the ReAct loop — running inside a block you can drag and drop

Gmail Trigger

↓

AI AGENT NODE

System Prompt:

```
You are a research assistant. Summarise the email and find
relevant background information. Keep your output concise and
factual.
```

Tools: Web Search | Wikipedia

↓

Slack — Send Output

1 The system prompt is the configuration

A few sentences of plain English. That's it. No logic flows. No decision trees.

2 This IS the ReAct loop

See → Think → Do → Check. The same loop from Slide 8 — now running inside a block you drag onto a canvas.

3 You're writing instructions, not code

If you can brief a smart colleague, you can write a system prompt. That's the whole job.

You're not configuring an algorithm. You're writing instructions — in plain English.

Building It Live — Together

WHAT WE'RE BUILDING: *A daily AI news digest delivered to Slack — "Every morning at 8am, find the top 3 AI stories from the last 24 hours, summarise each in two sentences, and post to #general."*

STEP 1	`Schedule Trigger`	→	**Set to fire every day at 8am. One click. This is what wakes the workflow up.**
STEP 2	`HTTP Request — News Source`	→	**Pull headlines from an RSS feed. This block fetches the content; it doesn't understand it yet.**
STEP 3	`AI Agent Node`	→	**Write the system prompt: "You are a news editor. Find the top 3 AI stories. Write a 2-sentence summary of each."**
STEP 4	`Slack Node`	→	**Connect to #general. Map the AI Agent output to the message field. One dropdown, one click.**
STEP 5	`Test It Live`	→	**Click "Test Workflow." Watch it execute. See the Slack message appear.**

Watch for Step 3 — the system prompt is where all the intelligence lives. What you write there shapes everything the agent does.

Your Turn to Think Like an Agent Builder

Two minutes. One task. First idea wins.

The Activity

2 MIN

Think of one thing in your work you do repeatedly — every day, every week, every time a certain thing happens.

1. Write it down in one sentence
2. Don't overthink it — first idea wins
3. Share with the room when time is up

This sentence is your system prompt. The rest is just connecting the blocks.

Your Template

"I want an agent that __________ every time __________ happens."

EXAMPLES FROM OTHER TEAMS

"Summarise every client email and add it to Notion."

"Alert Slack when a high-value lead fills our form."

"Draft a weekly report from our analytics data."

We'll sketch the n8n blocks for whatever you share with the room.

What You Now Know About n8n

01

You can build this — without code

Drag blocks. Connect them. Write a prompt in plain English. Free account. This week. An afternoon.

02

The AI Agent node IS the ReAct loop

Same reasoning engine as LangChain. Same See → Think → Do → Check. Just a visual interface on top.

03

It connects to the tools you already use

Gmail, Slack, Notion, Sheets, WhatsApp. You’re not adopting a new ecosystem — you’re putting automation between apps you’re already in.

WHERE TO GO NEXT

n8n.io

Sign up free. 900+ templates. Find one close to your use case and adapt it.

Start small

One trigger. One AI node. One output. Test it. Then add more.

Your use case

Write “I want an agent that...” and fill in the rest. That sentence is your system prompt.

If your first workflow saves you 15 minutes a week, that’s 13 hours a year. That’s worth building.

Everything we’ve built so far works silently. In the background.

AI Agents background ✓	**n8n** background ✓	**Ultravox** you talk to it

What if your agent had a voice?

> *Same loop. Same tools. Same memory.*
> *New interface: conversation.*

A Voice Agent Is an AI You Can Actually Talk To

Not a phone menu. Not transcription with a voice layer. A genuinely different kind of AI.

	Chatbot	Voice Agent (Ultravox)
Input method	You type	You speak
What it understands	Words only	Words + tone + timing + intent
Architecture	ASR → LLM → TTS (3 separate models)	Single speech-native model — no translation layer
Response latency	~1–2s per turn (each step adds delay)	Sub-200ms — near-human response speed
Interruption	Not possible mid-reply	You can cut it off mid-sentence
Can use tools	Yes (with tools)	Yes — native function calling while talking
Deployment	Chat interface / web embed	Phone (Twilio/SIP), web, native app — bring your own telephony

Ultravox doesn't convert your words to text first. It understands speech the way a person does — words, tone, timing, intent, all at once.

Why Most Voice AI Feels Robotic — and How Ultravox Fixes It

The architecture difference is the whole product.

TRADITIONAL PIPELINE

ASR Speech to Text ~400ms

↓

LLM Language Reasoning ~600ms

↓

TTS Text to Speech ~300ms

Total per turn: ~1.3 seconds — every single turn.

The polite pause you hear? That's the pipeline waiting.

VS

ULTRAVOX: SPEECH-NATIVE

Ultravox Speech-Native Model

Understands directly:

Words | Tone | Timing | Intent

No ASR stage | **No translation layer**

`Sub-200ms per turn`

AUDIO IN → **→ AUDIO OUT**

✓ Open-weight model — Llama 3, Qwen3, Gemma (v0.6)

✓ Built-in tool calling while in conversation

✓ 30+ languages, bring-your-own telephony

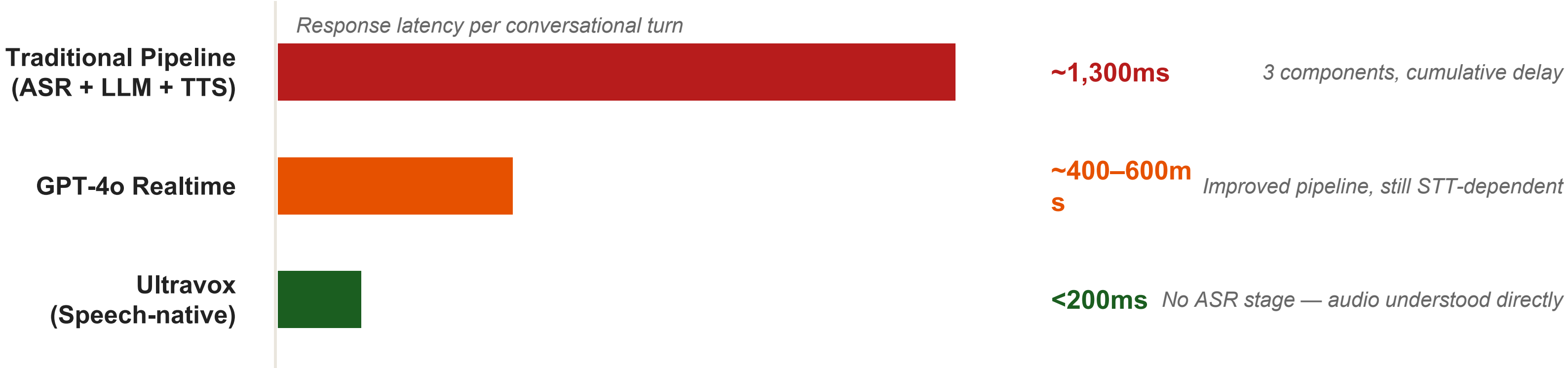
Audio in. Audio out. One model. No stitching.

✕ 3 components — 3 failure points ✕ Tone, emotion, timing lost in translation

Latency is the difference between a useful voice agent and an annoying one. Ultravox solves it at the architecture level.

Why Ultravox? The Numbers and What They Mean in Practice

Latency isn't a nice-to-have in voice AI — it's the product.



Users tolerate ~500ms before a voice response starts feeling unnatural. Ultravox stays well inside that threshold.

THREE THINGS ULTRAVOX HANDLES WELL

Natural Interruption

You can cut it off mid-sentence and it adapts immediately. It doesn't finish its sentence anyway.

Pause Handling

It knows whether you've stopped because you're done speaking or because you're still thinking.

Context Retention

Remembers what you said three exchanges ago. You never have to repeat yourself mid-call.

Real Uses. Running Today.

The pattern is the same: repetitive, phone-based work that currently requires a human

Customer Support

TASK

Handle tier-1 support calls automatically.

HOW IT WORKS

- Answers immediately — no hold music, no phone menu
- Reads the customer's account history in real time
- Resolves the issue or transfers to a human
- Attaches a full call summary to the ticket before handoff
- Human never has to ask "How can I help you?"

Available 24 / 7

No shift changes. No queue.

Appointment Booking

TASK

Book appointments through a natural phone call.

HOW IT WORKS

- Caller speaks naturally: "I'd like a haircut Thursday"
- Agent checks real-time availability
- Confirms the time slot in the same call
- Sends SMS confirmation automatically
- Nobody at the business picks up the phone

45 seconds, start to finish

Nobody typed anything.

Sales Qualification

TASK

Qualify leads before your sales team calls them.

HOW IT WORKS

- Agent calls leads from your CRM automatically
- Has a natural qualifying conversation
- Asks the same questions your team would ask
- Scores the lead and updates the CRM record
- Cold outreach — the part everyone hates — is gone

Sales team only talks to warm leads

11x built exactly this on Ultravox.

These aren't experimental. Companies are running all three in production today.

Let's Talk to One — Live

Pre-built on Ultravox. Same architecture. Real conversation.

The Agent

```
Name:           Nova
Platform:       Ultravox Realtime
Voice:          Emma
Role:           Receptionist, BrightStart Coaching
System prompt:
Understands:    Speech directly — no ASR stage
"You answer questions about BrightStart's services, collect
interest from callers, and offer to book an intro call. You
are warm, professional, and concise."
```

Nine sentences of English. That's the entire configuration. No code.

What to Listen For

Does it sound natural?
Or does it sound like a phone menu? Notice the flow of the conversation.

How fast does it respond?
Count the gap after you stop speaking. That's Ultravox's speech-native latency in real time.

Can it handle follow-ups?
Questions it wasn't explicitly given answers to — watch how it reasons.

The curveball
We'll ask something unexpected. Does it make something up or handle the edge case gracefully?

You're not evaluating it. You're experiencing it. Notice what surprises you.

Live — Nova, BrightStart Coaching

Conversation script — facilitator reference only. Follow the flow; don't read it out loud.

OPENING	"Hi, I'd like to learn more about BrightStart's coaching services."	→	Notice the response gap. That's the latency. Under 200ms on Ultravox.
FOLLOW-UP	"What kind of results do your clients typically see?"	→	Not in her briefing. She's reasoning from context, not reading a script.
DIG	"How long does the programme usually take?"	→	Watch it maintain persona and context across multiple turns.
CURVEBALL	"Actually — do you offer anything for teams, not just individuals?"	→	She wasn't told about team packages. Does she invent one or handle it gracefully?
CLOSE	"I think I'm interested. What's the next step?"	→	She should offer to book an intro call — her stated goal from the system prompt.

DEBRIEF — ask these after:

- **What surprised you?**
- **Where did it feel most natural?**
- **What would you change in the system prompt?**

You Can Build One of These

Here's how simple the Ultravox configuration actually is

ultravox console

```
name:             "Nova"

platform:         "Ultravox Realtime"

model:            "ultravox-v0.6"

voice:            "Emma"

architecture:     "speech-native // no ASR stage"

system_prompt:    "You are a warm, professional receptionist

      for BrightStart Coaching. Answer questions,

      collect interest, offer to book an intro call.

      Keep responses concise. Be human."

tools:            ["web_search", "calendar"]  // optional
```

1 The model IS the whole stack

Ultravox v0.6 understands speech directly. No separate ASR, no translation layer. Audio in, reasoning out.

2 The system prompt is the configuration

Four sentences of English. That defines who she is, what she does, and how she behaves. Nothing else needed.

3 Tools are optional and additive

Add web search, calendar, CRM — or run without any. The agent works either way. Tools extend it, they don't define it.

YOUR TURN — 2 minutes

"You are a ______ for ______. Your job is to..."

Here's What You've Just Learned

Three tools. One loop.
The same idea expressed three different ways.

n8n

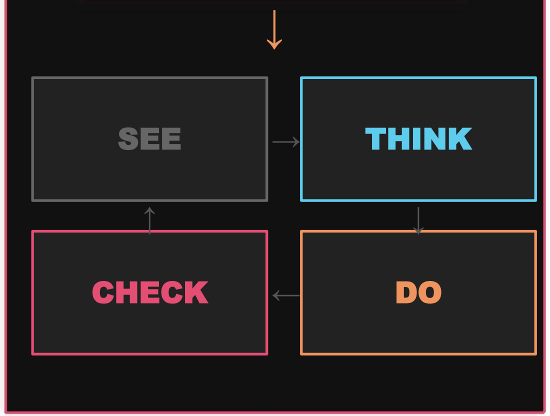
Visual — no-code

- Triggers + Actions
- AI Agent node
- 400+ integrations

AI Agents

Background — automated

- LangChain / Python
- Tools + Memory
- ReAct loop



Ultravox

Real-time — conversational

- Speech-native model
- No ASR stage
- Voice in / voice out

Background (automated)	Background (visual)	Real-time (conversational)

← automated conversational →

These aren't three different technologies. They're three expressions of the same idea: AI that acts.

If You Remember Nothing Else — Remember These

1

An agent is a loop

See → Think → Do → Check

It's not magic. It's not a black box. It's a loop with tools and a goal.

The brain reads the situation. It picks a tool. It runs it. It checks the result. It either stops or goes again.

Every agent you will ever encounter — regardless of what framework built it — is this loop.

If you understand this loop, you understand every agent.

2

You can build one

n8n — free — this week

n8n is free. It has 900+ templates. You don't need to understand everything before you start.

One trigger. One AI Agent node. One output. Test it. See what breaks. Then add one more thing.

You need to start to understand anything.

The barrier between here and a working agent is one afternoon.

3

You can talk to one today

Ultravox playground — free — 5 minutes

Go to the Ultravox playground. Pick a pre-built agent. Have a five-minute conversation.

Notice the latency. Notice how it handles questions it wasn't briefed on. Notice what you'd change.

That noticing is the beginning of building.

No setup. No account needed. Go now.

Thank you
AI for every human.

Session 3 — Understanding AI Agents

theaifellowship.com
community.theaifellowship.com